

# Adaptive Neural Caching for Low-Latency Query Execution

Author: Dr. Alex Vance, Systems Research Lab, MIT

Date: September 2026

## Abstract

Modern cloud databases suffer from high tail latencies due to inefficient query result caching. In this paper, we propose NeuroCache, an adaptive neural caching framework that predicts query re-use using lightweight embedding representations. Unlike traditional LRU and LFU heuristics, NeuroCache dynamically optimizes eviction policies using reinforcement learning. Our evaluation demonstrates an 18.4% improvement in cache hit ratio and a 34.2% reduction in P99 query latency on the TPC-H 100GB benchmark suite.

## 1. Introduction

Data-intensive enterprise applications demand sub-millisecond response times. Traditional cache eviction algorithms like Least Recently Used (LRU) fail to account for query computational cost and temporal query skew. When an expensive aggregation query is evicted prematurely, the database must recompute results from scratch, incurring severe CPU and I/O bottlenecks.

To solve this problem, we present a novel caching mechanism based on neural predictive scoring. Our primary contributions are:

- A compact neural feature representation for SQL query ASTs.
- An online actor-critic policy that balances hit rate against computation cost.
- Extensive empirical evaluation across synthetic and real-world database traces.

## 2. NeuroCache System Architecture

The NeuroCache pipeline consists of three core components: the Query Parser, the Embedder, and the Neural Eviction Engine. When a SQL query is received, its Abstract Syntax Tree (AST) is converted into a 64-dimensional dense vector representation.

### 2.1 Mathematical Formulation

The utility score  $U(q)$  of query  $q$  is calculated using the trade-off formula:

$$U(q) = \alpha \cdot C(q) + (1 - \alpha) \cdot P_{hit}(q) - \lambda \cdot S(q)$$

Where  $C(q)$  is the computational cost,  $P_{hit}(q)$  is predicted recurrence probability,  $S(q)$  is byte size, and  $\alpha$  is the weighting factor bounded by  $[0, 1]$ .

## 3. Experimental Evaluation & Results

Table 1: Performance comparison across 100,000 queries on TPC-H 100GB

| Method            | Hit Rate (%) | Avg Latency (ms) | P99 Latency (ms) |
|-------------------|--------------|------------------|------------------|
| LRU Baseline      | 54.2%        | 14.8 ms          | 128.5 ms         |
| LFU Baseline      | 58.1%        | 12.3 ms          | 112.0 ms         |
| ARC (Adaptive)    | 63.7%        | 9.7 ms           | 89.4 ms          |
| NeuroCache (Ours) | 75.4%        | 5.1 ms           | 42.6 ms          |

## 4. Limitations and Known Constraints

While NeuroCache delivers notable gains, several explicit limitations exist:

1. Cold-Start Overhead: During the first 500 queries, model predictions are unstable and fall back to standard LRU.
2. Memory Overhead: The neural model consumes an additional 128 MB of host RAM for storing embeddings.
3. Distributed Multi-Region Consistency: The current prototype is constrained to single-node deployments and does not handle cross-datacenter cache invalidations.

## 5. Conclusion

NeuroCache demonstrates that neural reinforcement policies can outperform conventional rule-based cache eviction algorithms in enterprise databases. Future work will investigate distributed consensus protocols and hardware-accelerated inference.